

AI Agent 系统设计

从经典计算机工程到现代智能体架构 - 第一部分：前九章完整版

Working Draft v0.5 - 2026-06-30

前言

这份文档不是一份 Agent API 使用手册，也不是一次聊天记录整理。它试图从软件工程师和分布式系统架构师的角度，解释为什么现代 AI Agent 的架构越来越像经典计算机系统。

本文的基本观点是：LLM 是最近几年 AI 的核心突破，但当 LLM 被放进真实产品和真实工作流之后，很多问题又重新回到了计算机工程最熟悉的领域：如何管理状态、如何降低远程调用成本、如何做缓存和索引、如何保持服务无状态、如何在多个计算资源之间调度。

第一部分先完成前九章：第 1 章建立整体视角；第 2 章把 LLM 放回“计算引擎”的位置；第 3 章说明 Agent 为什么更像 Orchestrator；第 4 章拆清 Memory、Tool 与 Planner 的职责边界；第 5 章讨论 Compute / Storage Separation；第 6 章说明 Stateless Agent 为什么接近微服务设计；第 7-9 章进一步讨论 Context Engineering、AGENTS.md 和 Retrieval / Context Routing。

核心主线：LLM 是新的 Compute Engine，Agent 是围绕它进行上下文管理、工具调用、状态恢复和资源调度的 Orchestrator。

目录大纲

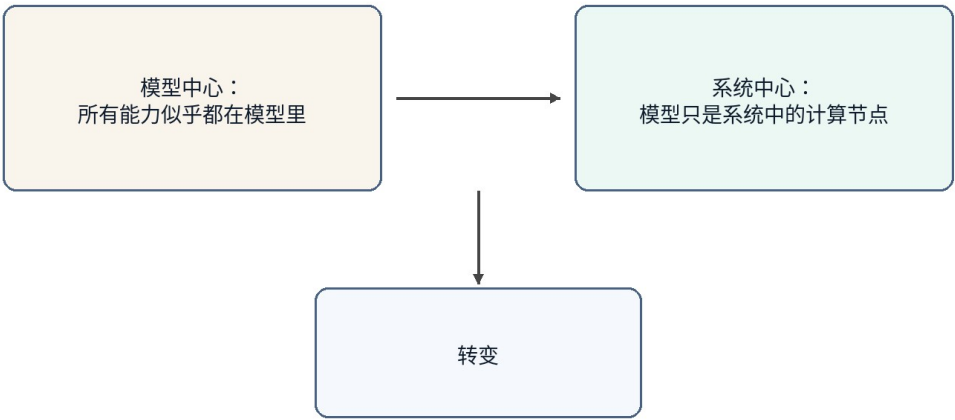
章节	标题	状态
第 1 章	为什么 AI Agent 让我想到了经典计算机工程	已完成
第 2 章	LLM：一种新的 Compute Engine	已完成
第 3 章	Agent：为什么它更像 Orchestrator	已完成
第 4 章	Memory、Tool 与 Planner 在 Agent 中的	已完成
第 5 章	Compute / Storage Separation 在 AI 中的体现	已完成
第 6 章	Stateless Agent 与微服务设计	已完成
第 7 章	Context Engineering 与 Query Optimization	已完成
第 8 章	AGENTS.md 与 Prompt Index	已完成
第 9 章	RAG、Retrieval 与 Context Routing	已完成
第 10 章	Token Reduction、Distillation 与 Tiered Compute	后续
第 11 章	Agent 生可靠性：等、待机与回放	后续

第 12 章	Multi-Agent、并发调度与多租户	后续
第 13 章	Agent 安全：Prompt Injection、Sandbox 与权限边界	后续
第 14 章	未来方向：Agent Operating System	后续

术语约定

术语	本文中的含义	工程类比
LLM	大言模型，推理算	Compute Engine
Agent	围绕 LLM 组织 Memory、Tool、Planner、Context 的系统层	Orchestrator / Runtime
Memory	期保存的用偏好、目背景和任状	Database / Cache
Context	本次求真正送入模型的工作集	Working Set / Buffer Pool
Tool	Agent 可调用的外部能力，例如文件、件、日、GitHub、Shell	RPC / API
Planner	拆解任务、排序步骤并决定是否继续、重试或升级的组件	Workflow Engine / Scheduler
Context Builder	从 Memory、文件、工具结果和任务状态中选择当前请求 working set 的组件	Query Optimizer / Buffer Manager
Context Routing	根据任务、权限、状态和成本选择信息源并构造上下文的过程	Query Planner / Router
Prompt Index	帮助 Agent 以低成本定位目知入口的索引结构	Database Index / Routing Table
Distillation	把大模型能力迁移到小模型或固定工作流	Precomputation / Tiered Compute
Sandbox	限制 Agent 工具、文件、网络 and 代行权限的隔离境	OS Process / Container
Idempotency	保证重试不会重复产生副作用的执行约束	Payment Idempotency / Exactly-once Boundary
Replay	复现 Agent 执行过程、工具调用和状态变化，用于调试、审计和对账	Event Log / Audit Trail

从模型中心到系统中心



图示为抽象架构，不代表某个产品的官方实现。

图 1

图 1 : AI Agent 的讨论正在从模型中心转向系统中心

第 1 章 为什么 AI Agent 让我想到了经典计算机工程

本章定位：建立从模型能力到系统架构的观察角度。

1.1 本章问题

过去几年，AI 的公众叙事通常围绕模型展开：模型更大了，推理更强了，编码更好了，长上下文更长了。这当然是事实，但如果只盯着模型，很容易忽略另一个正在发生的变化：AI 产品正在从“一个模型回答问题”变成“一个系统完成任务”。

一旦从回答问题转向完成任务，问题就不再只是模型能力，而是系统能力。系统需要保存长期状态，需要访问外部工具，需要理解当前工作目录，需要决定哪些历史信息应该进入上下文，需要在成本、延迟和正确性之间做权衡。这些问题并不陌生，它们正是经典计算机工程几十年来一直处理的问题。

1.2 核心观点

本章的核心观点是：AI Agent 并没有让软件工程过时，反而让很多经典软件工程思想重新变得重要。LLM 引入了一种新的高能力计算节点，但围绕这个节点构建可靠、可扩展、低成本系统，仍然需要分层、抽象、缓存、索引、调度、无状态、计算与存储分离等工程原则。

从这个视角看，Agent 不是单纯的“更聪明的聊天框”。它更像一个应用层运行时，负责把用户意图、长期记忆、项目文件、工具调用和模型推理组织成一个完整的执行过程。

1.3 传统计算机工程中的对应概念

经典计算机系统很少把所有能力都放在一个组件里。Web 服务通常把计算逻辑、缓存、数据库、消息队列、对象存储和外部 API 分开；分布式系统强调服务无状态、状态外置、请求可重试、资源可水平扩展；数据库系统强调索引、查询优化、buffer pool 和执行计划。

这些思想背后的共同目标是：不要让昂贵资源做不必要的工作。数据库不希望每次全表扫描；分布式系统不希望每次请求都跨多个服务；操作系统不希望每次访问都落到磁盘。今天的 AI Agent 也遇到了相似问题：不要把所有历史、所有文档、所有工具结果都塞给 LLM；不要把所有任务都交给最贵的大模型；不要让模型重复做已经可以被规则、小模型或缓存完成的事情。

1.4 AI Agent 中的实现形式

当我们把这些思想放到 Agent 里，会看到一组新的名字：Memory、RAG、Context Engineering、Tool Calling、Planner、Router、Distillation、MCP、Multi-Agent。它们听起来是 AI 时代的新概念，但背后的许多系统动机很熟悉。

Memory 像外部状态；Context Window 像一次请求的 working set；RAG 和检索像查询相关数据；AGENTS.md 像一个项目级 Prompt Index；Planner 像调度器；Tool Calling 像 RPC；MCP 像工具协议；Distillation 和小模型像把昂贵计算迁移到更低成本的计算层。名称改变了，但工程问题仍然是资源管理和系统优化。

1.5 不能过度类比的部分

类比有助于理解，但不能代替证明。AI Agent 和传统系统之间也有重要差异。传统 API 多数是确定性的，而 LLM 出具有概率性；存命中后返回固定，而小模型会生成果并可能犯；数据有明确 schema，而自然语言任务常常边界模糊。

所以本文不会说“Agent 就是数据库”或“LLM 就是 CPU”。更准确的说法是：LLM 成为新的计算对象之后，计算机工程里很多成熟的设计原则被重新应用到了 AI 系统中。类比的值在于帮助我们发现结构相似的问题，而不是把两个系统强行等同。

1.6 工程分析

从工程角度看，AI Agent 的关键挑战不是让模型单次回答更好，而是让整个系统持续、稳定、低成本地完成任。这意味着 Agent 需要做三类事情。第一类是上下文管理：决定哪些信息应该进入模型；第二类是资源调度：决定使用规则、小模型还是大模型；第三类是状态管理：决定哪些结果写回 Memory、文件系统或外部工具。

这三类事情本质上都是系统设计问题。它们决定了 Agent 能否从一次性的对话工具，升级成可以长期工作的数字系统。

1.7 本章小结

本章建立了全文的基本视角：AI Agent 的快速发展不是单纯依赖模型变强，而是模型能力与经典计算机工程思想结合的结果。LLM 是新的高能力计算节点，但 Agent 系统真正要解决的是如何围绕这个节点管理上下文、状态、工具和成本。

1.8 Agent 与经典计算机工程的快速对应

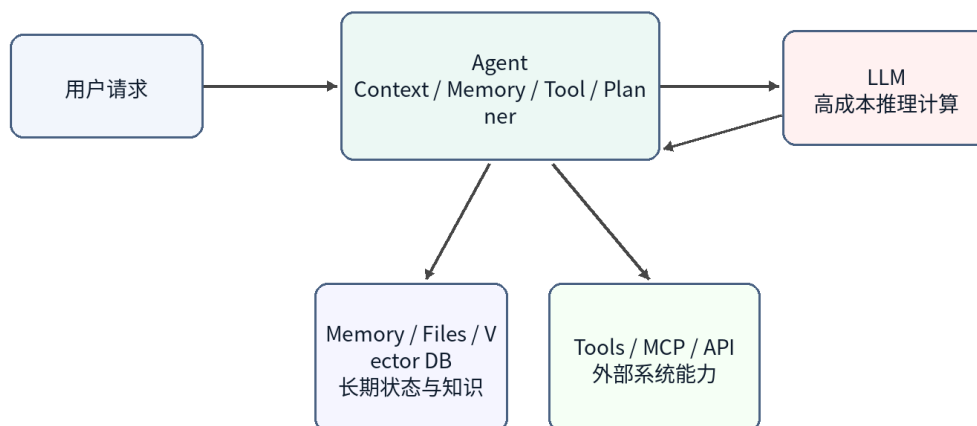
AI Agent 概念	典工程概念	说明
LLM	Compute Engine	负责高能力推理计算，但不应保存全部状态
Memory	Database / Cache	保存长期偏好、项目背景、任务历史
Context Window	Working Set / Buffer Pool	当前请求真正加载到计算层的数据
AGENTS.md	Index / Router	用小入口定位需要加载的项目知识
Tool Calling	RPC / API	调用外部软件系统完成真实操作
Planner	Scheduler / Workflow Engine	拆解任务并决定执行顺序
Small Model	Tiered Compute	低成本处理常见任务，复杂任务再升级

注意：这些对应关系用于帮助思考，不代表二者完全等同。

第 2 章 LLM：一种新的 Compute Engine

本章定位：把 LLM 放回计算节点的位置，而不是把它当成整个系统。

LLM 作为一种 Compute Engine



图示为抽象架构，不代表某个产品的官方实现。

图 2

图 2：LLM 作为 Agent 系统中的高成本计算引擎

2.1 本章问题

很多人在讨论 AI 系统时会把 ChatGPT、Claude、Gemini 这样的产品和底层 LLM 混在一起。实际上，产品不是模型本身。产品通常包含前端、Agent 层、Memory、工具调用、权限系统、文件系统、监控、计费和安全策略。LLM 只是其中最重要、也最昂贵的计算节点。

把 LLM 看成 Compute Engine，有助于我们重新理解 Agent 架构：模型不负责保存你的长期记忆，也不天然知道你的项目历史；它只是根据本次请求里的上下文进行推理。真正负责恢复状态、选择信息、调用工具的是 Agent 层。

2.2 LLM 的系统定位

在传统系统里，一个昂贵的计算服务通常不会直接暴露给所有业务逻辑随意调用，而是会被一层服务封装。比如搜索引擎背后有索引服务，数据库背后有查询优化器，GPU 推理服务前面可能有批处理、缓存和路由。

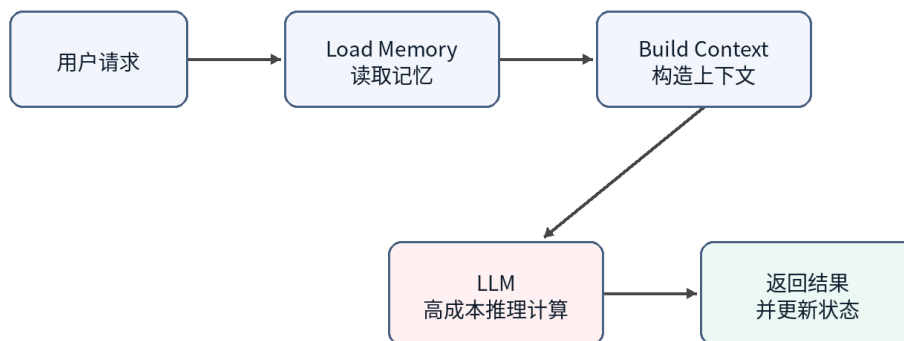
LLM 也应该这样理解。它可以处理自然语言、代码、推理和规划，但这并不意味着所有状态都应该放进模型，也不意味着所有任务都应该直接调用最大模型。LLM 更像一个非常强但成本很高的远程计算服务。每次调用都会产生输入 token、输出 token、延迟和 GPU 资源消耗。

2.3 Stateless Compute : 模型本身不保存会话状态

从一次推理请求的角度看，LLM 基本可以被视为 stateless compute。它不会因为上一轮和某个用户聊过，就在模型参数里永久记住这个用户。下一次推理时，如果上下文里没有提供相关信息，模型就无法可靠地知道之前发生过什么。

因此，所谓“模型记住了我”，在多数产品里并不是模型权重发生了个人化更新，而是 Agent 层把 Memory、近期对话、文件片段、工具结果重新拼接进 Prompt。模型看起来像记得，是因为每次请求前都被重新提供了必要状态。

Agent 的请求生命周期



图示为抽象架构，不代表某个产品的官方实现。

图 3

图 3：Stateless LLM 需要 Agent 在每次请求前恢复上下文

2.4 LLM API 是一种昂贵的远程调用

从工程成本看，调用 LLM API 很像调用一个昂贵的远程服务。它比普通函数调用慢，比大多数数据库查询贵，而且输出还不是完全确定的。因此 Agent 设计中一个非常核心的目标就是减少不必要的大模型调用。

这与传统系统优化非常类似。以前我们会减少跨服务 RPC、减少数据库查询、减少磁盘 IO；现在我们还要减少无意义的 LLM 调用、减少冗余上下文、减少重复推理。Token、上下文长度、模型层级和调用次数，正在成为 AI 系统新的性能指标。

2.5 Token 与 Compute Cost 不是同一个问题

这里需要区分 token 数量和计算成本。这个区分很容易被讲错。比如有人会说“通过模型蒸馏减少 token 使用”，但严格说，蒸馏主要减少的不是 token 数量，而是处理同样 token 所需要的算力成本。一个小模型和一个大模型可能吃进去同样多的 token，但小模型的单 token 成本更低。

可以把 Agent 的模型调用成本拆成一个简单公式：

总成本 ≈ 调用次数 × 单次 token 量 × 单 token 成本

这三个因子对应三组不同的优化。Planner、缓存命中和任务合并主要减少调用次数；Context Engineering、RAG、裁剪、摘要、Prompt Caching 和工具输出压缩主要减少单次 token 量；蒸馏、小模型、路由和级联主要减少单 token 成本。

维度	降 Token 量	降单 Token 算力成本
优化对象	每次求送和吐出的 token 数	理同 token 所耗的算力和价
分布式类比	减少 RPC、缩小 payload、减少往返	把任务下沉到更便宜的计算层，例如冷热戴服等级
典型手段	Context Engineering、RAG、裁剪、摘要、Prompt Caching、压缩工具输出、Planner 迭代上限	蒸馏、小模型、路由、级联，先用便宜模型尝试，低置信再升级
是否需要训练和数据	通常不需要，工程和配置即可	蒸馏需要标注、训练、评估和部署；路由或现成小模型不一定需要
落地成本和见效速度	低成本、见效快，改 prompt、检索和缓存策略就可能有效	蒸馏成本高；路由中等；收益依赖任务稳定性和评估体系
主要	裁剪上下文表，模型基于不完整信息回答	路由错档，简单任务上大模型会浪费，复杂任务下放小模型会出错
不确定性来源	检索召回质量、摘要保真度、工具输出压缩是否损坏信息	小模型泛化边界、置信度估计是否可靠
独立开发优先级	先做，门槛低，几乎不需要管	稳定流量和数据积累之后再考虑，蒸馏不是起手式

因此更准确的说法是：蒸馏主要减少对昂贵大模型的使用，而不是必然减少 token。只有当蒸馏让工作流从多次大模型调用变成一次小模型处理，或者减少了多轮规划过程时，token 总量才可能随之下降。此时下降的是被省掉的调用和中间上下文，而不是“蒸馏”直接压缩了每次请求里的 token。

这个区分对真实产品很重要。对独立开发者来说，蒸馏是重武器：需要标注数据、训练管线、评估集、部署和模型托管。相比之下，减少 token 的那组手段更像常规系统优化：缓存稳定前缀、裁剪无关上下文、压缩工具输出、限制 Planner 迭代次数、用路由避免无意义的大模型调用。这些通常不需要训练，改工程结构就能见效。

所以如果目标是先把成本降下来，更现实的顺序通常是：先榨干调用次数和 token 量，再考虑路由、小模型和蒸馏。蒸馏适合放在最后，尤其适合那些已经有稳定流量、稳定任务分布和足够样本的工作流。

2.6 与传统 Compute 的类比

LLM 作为 Compute Engine，可以类比数据库里的执行引擎、云上的远程计算服务、或者 GPU 推理集群。它的能力强，但不应该承担系统中的所有职责。就像数据库不会负责业务流程，CPU 不会负责操作系统调度，LLM 也不应该被看成整个 Agent 系统本身。

这个视角让我们更容易理解为什么 Agent 层重要。Agent 的任务不是取代模型，而是让模型在正确的时间、拿到正确的上下文、以合理的成本完成计算。

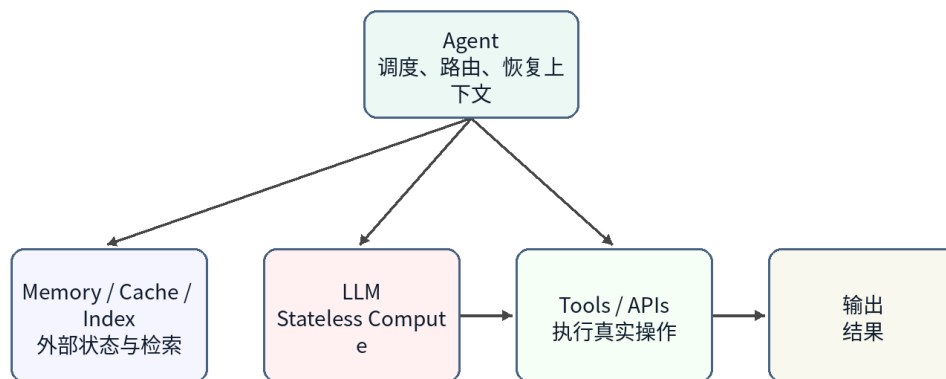
2.7 本章小结

本章把 LLM 定位为新的高能力 Compute Engine。它强大、昂贵、基本无状态，并且通过 API 被 Agent 调用。这个定位解释了为什么 Memory、Context、Tool、Planner 等能力不应该被简单归入模型本身，而应该被视为 Agent 系统的组成部分。

第 3 章 Agent：为什么它更像 Orchestrator

本章定位：Agent 是调度和编排层，负责让合适的资源处理合适的任务。

Agent 作为 Orchestrator



图示为抽象架构，不代表某个产品的官方实现。

图 4

图 4：Agent 更像 Orchestrator，而不只是聊天机器人

3.1 本章问题

如果 LLM 是 Compute Engine，那么 Agent 是什么？一个常见误解是把 Agent 理解成“大模型的外壳”或者“带工具的聊天机器人”。这个说法抓住了一部分现象，但没有抓住架构本质。

更准确地说，Agent 是一个 Orchestrator。它负责接收用户目标，恢复相关状态，选择需要的上下文，决定是否调用工具，判断使用哪个模型，并把多步过程组织成一个可执行的任务流。

3.2 Agent 的核心组成

一个典型 Agent 至少包含几个部分。Memory 保存期偏好、目背景和任史；Context Builder 决定哪些信息进入当前 Prompt；Planner 把用户目标拆成步骤；Tool Layer 负责调用外部系统；Router 或 Scheduler 决定使用规则、小模型还是大模型；Evaluator 或 Guardrail 用来判断结果是否足够可靠。

这些组件可以在物理实现上分布在多个服务中，也可以被封装在一个产品内部。但从逻辑架构看，它们共同构成 Agent，而不是 LLM 的一部分。

3.3 Agent 的典型执行流程

一个 Agent 接到请求后，通常不会立刻把用户原文扔给大模型。更合理的流程是：先识别任务类型，再读取相关 Memory 或文件，再构造上下文，然后选择计算资源。如果是简单规则可以处理

的任务，就不用模型；如果是常见模式，可以交给小模型；如果复杂度高或风险高，再升级到大模型。

这就是 Agent 与普通聊天机器人的差别。聊天机器人偏向“输入一句话，输出一句话”；Agent 偏向“接收一个目标，组织一组计算与操作来完成它”。

3.4 Agent 与 API Gateway / Scheduler 的关系

从系统类比看，Agent 有点像 API Gateway、Workflow Engine 和 Scheduler 的混合体。它对外暴露统一入口，对内连接多个资源：模型、工具、Memory、文件、外部服务。它还需要根据任务复杂度和成本选择执行路径。

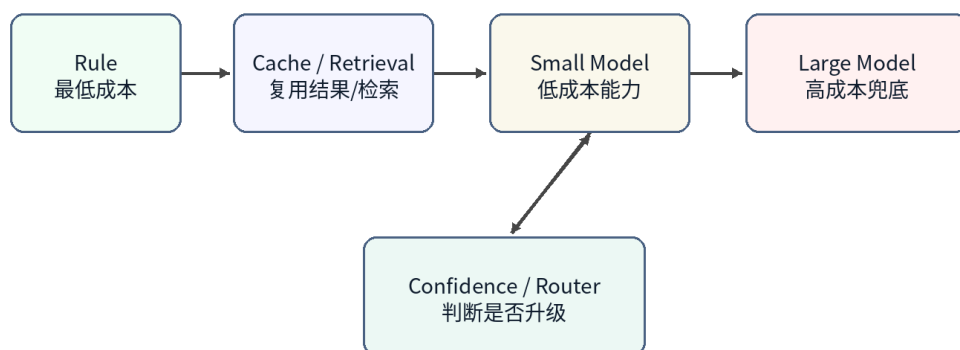
这正是 Orchestrator 的职责：它不一定自己完成所有计算，但它决定哪些资源参与、以什么顺序参与、结果如何合并，以及失败时是否重试、降级或升级。

3.5 分层计算：Rule、Cache、Small Model、Large Model

Agent 的资源调度可以进一步抽象为分层计算。最低层是规则，成本最低、速度最快，但覆盖范围有限。下一层是缓存或检索，可以复用已有结果或找到相关上下文。再往上是小模型，它不像缓存那样保存固定答案，而是保存通过蒸馏获得的能力。最高层是大模型，能力最强、成本最高，适合处理复杂和未知问题。

这个结构和传统系统中的多级缓存、冷热分层、服务降级非常相似。真正优秀的 Agent 不应该让每个请求都直接落到最昂贵的大模型上，而应该尽量在便宜层解决问题。

分层计算：从便宜资源到昂贵模型



图示为抽象架构，不代表某个产品的官方实现。

图 5

图 5：分层计算让 Agent 避免所有请求都落到大模型

3.6 小模型为什么不是普通 Cache

小模型经常被类比成缓存，但这个类比需要修正。普通缓存保存的是结果，命中后直接返回；小模型保存的是能力，它会根据输入重新推理。它不能保证像 Redis 一样返回固定值，但它可以泛化到训练集中没有出现过的新问题。

所以更准确的说法是：蒸馏后的小模型像一种 capability cache，或者说是把大模型在大量样本上表现出的能力压缩成较低成本的计算层。它不是消灭计算，而是把计算从昂贵模型迁移到便宜模型。

3.7 Agent 的局限与风险

Agent 作为 Orchestrator 也带来新的风险。第一，路由错误会导致简单任务被过度调用大模型，或者复杂任务被错误交给小模型。第二，Memory 索引模型基于上下文回答。第三，多步工具调用会放大局部错误。第四，概率模型的不确定性使传统测试方法不完全适用。

因此 Agent 系统需要可观测性、日志、审计、回放、评估集和升级策略。这些再次说明 Agent 已经不是一个单纯的对话界面，而是一个需要系统工程方法来设计和运维的软件系统。

3.8 本章小结

本章将 Agent 定位为 Orchestrator。它不是模型本身，而是围绕模型组织 Memory、Tool、Planner、Context 和资源调度的系统层。这个定位为后续章节铺垫了基础：Memory 为什么像 Storage，AGENTS.md 为什么像 Index，Distillation 为什么像 Tiered Compute，以及 Multi-Agent 为什么会越来越接近分布式系统。

第 4 章 Memory、Tool 与 Planner 在 Agent 中的职责

本章定位：拆清 Agent 内部三个最容易混在一起的职责边界。

4.1 本章问题

前面几章把 Agent 定位为 Orchestrator，但只说 Agent 负责组织 Memory、Tool、Planner 和 Context 还不够。真正做系统设计时，最容易出问题的地方不是“有没有这些模块”，而是这些模块之间的职责边界不清。

很多 Agent 产品会把所有东西都塞进一个大 Prompt：用户历史、项目背景、工具结果、计划步骤、错误信息和最终回答都混在一起。短期看这样可以跑通 Demo，长期看会导致三个问题：状态不可控，工具调用不可审计，规划过程不可复现。

因此，本章要回答的问题是：Memory 应该保存什么，Tool 应该负责什么，Planner 又应该决定什么？这三个组件分别对应传统系统中的 Storage、外部 API 和 Workflow / Scheduler，但它们不是同一个东西。

4.2 Memory：保存状态，而不是替模型思考

Memory 的职责是保存可复用状态。它可以包括用户偏好、项目背景、历史决策、任务进度、文件摘要和长期事实。它不应该被理解成“让模型变聪明”的魔法层，而应该被理解成 Agent 系统外置的状态存储。

从工程角度看，Memory 更像 Database / Cache，而不是模型的一部分。模型每次推理时仍然是 stateless compute。Agent 需要先从 Memory 中读取相关内容，再把其中一部分放进当前 Context。也就是说，Memory 存得多不代表模型看得多，真正进入模型的是 Context Builder 选择后的 working set。

这一区分很重要。如果 Memory 设计得像一个无限增长的聊天记录，系统会很快退化成“把历史全塞进 Prompt”。更合理的做法是把 Memory 分层：长期稳定事实单独保存，近期任务状态单独保存，临时工具结果只在任务生命周期内保留，需要长期复用的结果再经过整理后写回。

4.3 Tool：产生副作用，而不是保存长期语义

Tool 的职责是连接外部世界。它可以读取文件、发邮件、查日历、访问 GitHub、执行 Shell、调用数据库或操作业务系统。Tool 的核心特征是：它可能产生真实副作用。

这使 Tool 和 Memory 有本质区别。Memory 是 Agent 自己的状态；Tool 是外部系统能力。一次工具调用可能创建 issue、发送消息、修改文件、发起支付、更新订单或删除资源。这些操作不能只当成“模型输出的一部分”，而必须当成可审计、可重试、可回滚或至少可解释的系统行为。

因此，Tool Layer 至少需要处理四件事：权限、参数校验、执行结果、错误语义。Agent 不能只让模型生成一段自然语言，然后模糊地说“我调用了工具”。它需要记录调用了哪个工具、传入了什么参数、返回了什么结果、是否有副作用、失败后是否可以重试。

4.4 Planner：决定步骤，而不是直接承担执行

Planner 的职责是把用户目标拆成可执行步骤，并决定步骤之间的顺序。它回答的是“接下来应该做什么”，而不是“工具内部如何做”或“状态应该如何持久化”。

这和传统系统中的 Workflow Engine 或 Scheduler 类似。一个工作流系统不会自己发邮件、写数据库或跑查询；它决定这些操作什么时候发生、依赖什么前置条件、失败后是否重试、是否需要人工确认。Agent Planner 也应该类似：它负责组织任务，而不是把所有业务逻辑都写进 Prompt。

一个常见错误是让 Planner 过度自由。模型可以生成很长的计划，但计划越长，失败点越多，成本越高，越难审计。更稳妥的做法是让 Planner 生成短计划，并在每一步之后根据工具结果重新评估，而不是一次性规划十几步。

4.5 Context Builder：三者之间的连接层

Memory、Tool 和 Planner 之间还需要一个容易被忽略的组件：Context Builder。它负责把长期状态、当前任务、工具结果和计划步骤整理成当前模型调用所需的 Context。

Context Builder 不是简单拼字符串。它要决定哪些 Memory 相关，哪些工具结果需要保留，哪些计划步骤已经完成，哪些错误需要提醒模型，哪些信息应该被压缩或丢弃。它本质上像数据库查询优化器和操作系统 working set 管理的混合体。

如果没有 Context Builder，Memory 会变成一堆不可控文本，Tool Result 会变成不断膨胀的日志，Planner 会变成没有反馈机制的长列表。Agent 的可靠性很大程度取决于这层是否清楚地管理“本次请求到底让模型看到什么”。

4.6 三个组件的职责边界

组件	主要职责	不应该承担的职责	工程类比
Memory	保存长期状态、偏好、项目背景和任务历史	直接替模型推理，或把所有历史无差别塞进 Prompt	Database / Cache
Tool	调用外部系统并返回结果，必要时产生副作用	保存长期语义，或隐藏真实副作用	RPC / API
Planner	拆解目标、排序步骤、决定是否继续或升级	直接执行外部操作，或一次性生成不可调整的长计划	Workflow Engine / Scheduler
Context Builder	选择当前请求的 working set	无限制拼接所有信息	Query Optimizer / Buffer Manager

这张表的重点不是给组件起名字，而是避免职责泄漏。Memory 不应该偷偷变成 Prompt 垃圾桶；Tool 不应该被当成无副作用函数；Planner 不应该变成不可审计的自然语言脚本；Context Builder 不应该只是字符串拼接器。

4.7 失败模式

职责边界不清会带来一组典型失败模式。

第一，Memory 污染。错误信息、过期事实或临时工具结果被写入长期 Memory，导致后续任务反复引用错误上下文。

第二，Tool 副作用失控。模型重复调用同一个工具，创建重复 issue、重复发送消息或重复修改文件，而系统没有幂等键和调用日志。

第三，Planner 失控。模型生成过长计划，执行过程中没有检查点，也没有根据工具结果更新路径，最后失败时无法判断哪一步出了问题。

第四，Context 膨胀。所有历史和工具结果都被塞进模型，token 成本上升，关键事实反而被噪音淹没。

4.8 本章小结

本章把 Agent 内部的 Memory、Tool、Planner 和 Context Builder 拆开。Memory 是状态层，Tool 是外部能力层，Planner 是任务编排层，Context Builder 是当前请求的 working set 管理层。

这个划分为后续章节铺垫了基础：第 5 章会进一步讨论为什么 Compute 和 Storage 应该分离；第 6 章会说明为什么 Agent 本身应该尽量保持 stateless；后面的生产可靠性章节会回到 Tool 副作用、幂等、状态机和回放。

第 5 章 Compute / Storage Separation 在 AI 中的体现

本章定位：说明为什么 Agent 架构不应该把模型、状态和知识混成一个整体。

5.1 本章问题

传统系统设计里，一个重要原则是计算与存储分离。计算层负责执行逻辑，存储层负责保存状态。Web 服务不会把所有用户状态放进进程内存；数据库不会把所有业务流程塞进查询执行器；分布式系统也不会假设某个计算节点永远持有完整上下文。

AI Agent 也会遇到同样问题。LLM 是高能力 Compute Engine，但它不是长期存储层。Agent 可以调用模型、读取 Memory、检索文档、调用工具、更新状态，但如果把这些职责全部塞进 Prompt 或模型权重里，系统很快会变得昂贵、脆弱且不可维护。

本章的核心问题是：在 Agent 系统中，哪些东西应该被看成 Compute，哪些东西应该被看成 Storage？为什么这两者分离后，Agent 才更容易扩展、调试和控制成本？

5.2 LLM 是计算层，不是存储层

LLM 的职责是根据当前输入进行推理、生成和判断。它可以处理复杂语言、代码和规划任务，但它不应该被当成保存所有历史和业务状态的地方。

从一次调用看，LLM 更像远程计算服务。它读取 Prompt，执行推理，返回结果。下一次调用时，如果 Prompt 中没有提供相关状态，它不能可靠地恢复之前发生过什么。即使模型有很长上下文窗口，这也不等于它适合作为数据库。长上下文只是更大的 working set，不是持久化存储。

因此，“把所有东西塞进上下文”不是计算与存储分离，而是把存储临时搬进计算请求。它可以解决少量临时任务，但不适合长期系统。

5.3 Memory、文件和索引是存储层

Agent 系统里的存储层可以有多种形态：结构化数据库、向量索引、文件系统、对象存储、缓存、事件日志、用户偏好表、任务状态表等。它们共同的职责是保存状态，并允许 Agent 在需要时读取。

Memory 只是存储层的一部分。项目文件、工具执行结果、用户配置、权限策略、任务进度和审计日志都属于更广义的 Storage。把它们都叫 Memory 会模糊边界。更准确的说法是：Memory 是面向模型上下文恢复的状态视图，而 Storage 是系统真实持久化状态的集合。

这一区分会影响架构设计。比如，用户偏好可以存入 Memory；订单状态应该存入业务数据库；工具调用日志应该进入审计日志；大文档应该进入文件系统或对象存储；用于检索的片段应该进入索引。这些东西不应该全都变成一段 Prompt 文本。

5.4 Context 是计算请求的 working set

Compute / Storage Separation 并不意味着计算层不需要状态。相反，每次计算都需要一个经过选择的状态子集，这就是 Context。

Context 像数据库查询时加载到 buffer pool 的数据页，也像操作系统进程当前使用的 working set。它来自 Storage，但不是 Storage 本身。Agent 每次调用模型前，需要从 Memory、文件、索引和工具结果中选择一小部分，把它们组织成模型可以处理的输入。

这解释了为什么 Context Engineering 是 Agent 系统的核心能力。它不是简单 Prompt 技巧，而是从存储层到计算层的数据加载策略。加载太少，模型缺上下文；加载太多，成本高、噪音大、关键事实被淹没。

5.5 RAG 是读取路径，不是系统全部

RAG 经常被当成 Agent 架构的核心，但更准确地说，RAG 是一种从 Storage 到 Context 的读取路径。它通过检索把相关文档片段放进模型上下文，让 LLM 能基于外部知识回答。

这很有用，但它不是完整的状态管理。RAG 主要解决“从大量文本中找相关内容”的问题，不自动解决任务状态、权限、副作用、重试、审计、长期偏好和业务一致性。一个生产 Agent 不能只靠 RAG 管理所有状态。

因此，RAG 应该被放回系统架构里的正确位置：它是 Storage 读取路径的一部分，和数据库查询、文件读取、缓存命中、工具结果加载并列。Agent 需要决定什么时候用 RAG，什么时候查结构化状态，什么时候读取文件，什么时候调用工具。

5.6 写回路径同样重要

很多 Agent 讨论只关注“如何把信息读进 Prompt”，却忽略“哪些结果应该写回 Storage”。但对长期运行的系统来说，写回路径同样重要。

Agent 完成任务后，可能需要写回几类结果：用户偏好、任务进度、决策记录、工具调用摘要、错误信息、评估结果、审计日志。不是所有模型输出都应该写回。写回前需要判断信息是否稳定、是否真实、是否可复用、是否有权限保存。

这和传统系统里的写路径类似。数据库写入需要 schema、约束和事务；缓存写入需要过期策略；事件日志需要顺序和不可篡改性。Agent Memory 写回也应该有类似纪律，否则系统会把模型幻觉、临时推测和过期状态永久保存下来。

5.7 分离后的工程收益

设计问题	混在一起的做法	分离后的做法
长期状态	全部塞进 Prompt 或聊天页	存入数据库、文件、Memory 或事件日志
当前上下文	无限制拼接历史	从 Storage 中选择 working set
成本控制	长上下文不断膨胀	通过索引、缓存和裁剪控制 token
可调试性	不知道模型基于什么状态回答	可以查看读取了哪些 Memory、文件和工具结果
一致性	模型输出直接变成事实	写回前经过验证、结构化和权限检查
扩展性	每次请求都携带大量历史	存储独立扩展，计算层按需加载

Compute / Storage Separation 的最大收益是可控性。模型仍然很强，但它不再是系统里所有状态的容器。状态在哪里保存、什么时候读取、读多少、什么时候写回，都变成可以设计和审计的工程问题。

5.8 本章小结

本章把 AI Agent 中的计算与存储分开看。LLM 是计算层，Memory、文件、索引、数据库和日志构成存储层，Context 是每次计算请求的 working set，RAG 是一种读取路径，而不是完整系统。

这个视角解释了为什么 Agent 不能只靠长上下文解决一切。长上下文扩大了单次请求的 working set，但没有取代持久化状态、索引、写回策略和一致性控制。第 6 章会进一步说明：当状态外置后，Agent 本身就可以更接近 stateless service。

第 6 章 Stateless Agent 与微服务设计

本章定位：说明为什么 Agent 层应该尽量无状态，以及状态外置后系统如何变得更可靠。

6.1 本章问题

第 5 章讨论了 Compute / Storage Separation。本章继续往下推一步：如果 LLM 是 stateless compute，Memory 和文件是外部 Storage，那么 Agent 层本身应该是什么状态？

一个直觉做法是让 Agent 进程保存大量会话状态：当前计划、历史工具结果、用户偏好、执行进度、错误上下文都放在进程内存里。这样实现简单，但和传统微服务里的有状态服务一样，会带来扩展、重启、重试和故障恢复问题。

更稳妥的设计是：Agent 尽量做成 stateless service。它可以在一次请求中临时持有执行上下文，但长期状态应该外置到 Memory、数据库、文件系统、任务队列或事件日志中。这样 Agent 实例可以水平扩展、失败重试、替换和回放。

6.2 Stateless 不等于没有状态

Stateless Agent 并不是说系统没有状态，而是说状态不应该依赖某个 Agent 进程的内存。状态仍然存在，只是被放到了外部存储中，并通过明确的读写路径恢复。

这和 Web 服务非常类似。一个无状态 Web 服务并不是不处理用户登录、购物车或订单，而是 not 把这些状态只存在某台服务器的内存里。请求到来时，它从 Cookie、Session Store、数据库或缓存恢复状态；请求结束后，把必要结果写回外部系统。

Agent 也应该类似。每次任务执行前，Agent 可以读取用户 Memory、项目文件、任务状态和工具日志；执行过程中构造 Context；执行后把稳定结果写回。只要这些状态不绑定到某个进程，Agent 就可以更接近无状态服务。

6.3 为什么 Agent 容易变成有状态泥球

Agent 很容易滑向有状态泥球，因为自然语言任务本身边界模糊。模型上一轮说过什么、工具返回了什么、用户临时改了什么、Planner 走到哪一步，都看起来像“应该记住”的东西。

如果没有明确设计，最简单的实现就是把这些东西都放在一个运行时对象里，然后不断 append。短期看这很方便，长期看会造成几个问题：进程重启后任务丢失；多实例之间状态不一致；失败重试无法判断是否已经执行过某个工具；用户刷新页面后上下文恢复不完整；日志和真实状态对不上。

这也是为什么 Agent 系统不能只关注 Prompt。Prompt 是一次模型调用的输入，不是系统状态的唯一来源。真正的状态应该有结构、有生命周期、有写回策略。

6.4 外置状态的基本形态

Stateless Agent 需要把不同状态放到不同外部系统中。

状态类型	合适位置	说明
------	------	----

用户偏好	Memory / 用户配置表	长期稳定，但需要可编辑和可删除
目背景	文件系统 / 文档索引 / Memory 摘要	可检索，但不应全部塞进上下文
任务进度	任务状态表 / Workflow State	用于恢复执行、展示进度和失败重试
工具调用	审计日志 / Event Log	记录参数、结果、副作用和错误
临时上下文	请求内存 / 短期缓存	只在当前任生命周期内使用
最终产物	文件、数据库或外部业务系统	根据任务类型持久化

这个表的重点是生命周期。不是所有状态都应该长期保存，也不是所有状态都应该进入 Memory。临时上下文可以随着请求结束而消失；工具调用日志可能需要长期保留；用户偏好需要支持修改和删除；任务状态需要足够结构化，才能恢复和回放。

6.5 Stateless 带来的扩展性

当 Agent 本身无状态后，系统可以像普通微服务一样扩展。多个 Agent 实例可以处理不同请求，因为它们不依赖本地内存里的长期状态。实例挂掉后，新的实例可以从外部状态恢复任务。流量上升时，可以水平增加实例。升级模型或 Agent 代码时，也更容易滚动发布。

这对 Agent 尤其重要，因为 Agent 调用可能很慢。一次任务可能涉及多次模型调用、多次工具调用和等待外部系统。不能假设同一个进程永远在线，也不能假设一次请求会在一个短连接里完成。任务队列、状态表和事件日志会比进程内对象更可靠。

Stateless 还让多租户更简单。不同用户和项目的状态可以通过租户 ID、权限策略和存储隔离来管理，而不是依赖某个 Agent 进程“记得自己正在服务谁”。

6.6 Stateless 带来的重试和恢复能力

Agent 执行中最危险的地方是重试。模型调用失败可以重试，检索失败可以重试，但工具调用可能已经产生副作用。如果 Agent 状态只在内存里，系统很难判断一次失败发生在副作用之前还是之后。

外置状态可以让重试更可控。每个任务步骤可以有状态：pending、running、succeeded、failed、needs_review。每次工具调用可以有 idempotency key、参数摘要、结果摘要和副作用记录。重试前先检查状态，就能避免重复执行已经成功的操作。

这正是传统生产系统里的思路。支付系统、订单系统和消息队列都会把“是否已经执行”作为状态保存下来，而不是依赖进程记忆。Agent 一旦开始调用真实工具，也需要同样的纪律。

6.7 Stateless 的代价

Stateless 不是免费午餐。把状态外置后，系统需要更多基础设施：数据库、缓存、对象存储、任务队列、事件日志和权限系统。每次请求也需要先恢复状态，再构造 Context，延迟和复杂度都会增加。

此外，状态外置会暴露 schema 设计问题。哪些字段需要结构化？哪些内容只适合保存原始文本？工具结果应该保存全文还是摘要？Memory 写回要不要人工确认？这些问题不能靠“放进 Prompt”绕过去。

所以 Stateless Agent 的目标不是把所有东西都拆得很重，而是在可靠性和实现成本之间做取舍。早期产品可以从轻量状态表和简单日志开始；随着任务变复杂，再引入更严格的 Workflow State、审计日志和回放机制。

6.8 本章小结

本章把 Agent 层看成一种尽量无状态的服务。Stateless 不等于系统没有状态，而是状态不绑定在某个 Agent 进程里。Memory、任务状态、工具日志和最终产物都应该通过外部存储管理。

这个设计让 Agent 更容易扩展、重启、重试、恢复和审计。它也把我们带到后续章节：Context Engineering 会讨论如何从外部状态构造当前 working set；生产可靠性章节会进一步讨论幂等、状态机、回放和对账。

第 7 章 Context Engineering 与 Query Optimization

本章定位：把 Context Engineering 从 Prompt 技巧提升为数据加载和查询优化问题。

7.1 本章问题

前面几章已经把 LLM 定位为 Compute Engine，把 Memory、文件和索引定位为 Storage，把 Context 定位为一次计算请求的 working set。接下来最重要的问题就是：每次调用模型前，到底应该加载哪些信息？

很多人把 Context Engineering 理解成“把 Prompt 写得更好”。这当然是其中一部分，但从系统角度看，它更像 Query Optimization。数据库不会把整张表都送进执行器，操作系统也不会把整块磁盘都加载到内存。Agent 同样不应该把所有聊天历史、所有文档和所有工具结果都塞进模型。

Context Engineering 的核心不是“更多上下文”，而是“更相关、更便宜、更可验证的上下文”。它要在召回率、精度、成本、延迟和可靠性之间做权衡。

7.2 Context 是模型调用的执行计划输入

一次模型调用看起来像一个函数调用，但它的输入并不是用户原话，而是 Agent 构造出来的 Context。这个 Context 可能包含系统指令、用户目标、Memory 摘要、文件片段、工具返回、计划步骤、错误信息和格式要求。

因此，Context 更像数据库查询执行前的执行计划输入。执行器看到的不是原始业务世界，而是优化器选择后的数据和操作顺序。LLM 看到的也不是完整项目，而是 Context Builder 选择后的工作集。

这意味着上下文质量直接决定模型质量。模型可能很强，但如果 Context 缺少关键事实，它会猜；如果 Context 混入错误事实，它会被误导；如果 Context 太长，重要信息会被噪音稀释，成本也会上升。

7.3 Query Optimization 的类比

数据库查询优化器会估计哪些索引可用、哪些表需要 join、过滤条件选择性如何、扫描成本是多少。Context Builder 也需要做类似判断：哪些 Memory 相关，哪些文件片段应该加载，工具结果是否仍然有效，旧上下文是否可以丢弃。

这不是完美类比。数据库有 schema、统计信息和确定性执行计划；自然语言任务更模糊，相关性更难估计。但工程动机相同：不要把昂贵计算浪费在无关数据上。

如果把 LLM 看成昂贵执行器，那么 Context Engineering 就是在调用前做数据选择、过滤、排序和压缩。它决定模型看到什么，也决定模型不看到什么。

7.4 Context Builder 的输入来源

输入来源	典型内容	主要风险	优化问题
用户请求	当前目标、约束、偏好	表述模糊、目标变化	澄清意图和抽取任务

Memory	用户偏好、项目背景、历史决策	过期、污染、过度召回	选择稳定且相关的状态
文件系统	README、代码、文档、配置	文件过多、版本不一致	找到当前任真正相关的文件
Retrieval	文档片段、知识库内容	召回、片段断裂	平衡召回率和精度
Tool Result	API 返回、Shell 出、搜索结果	过长、临时、含错误	压缩并保留可验证事实
Planner State	当前步骤、完成情况、失败原因	计划过长、状态漂移	保留下一步所需的最小状态

这个表说明 Context 不是单一文本，而是多个来源的组合。Context Engineering 的难点在于跨来源选择，而不是只优化某一段 Prompt。

7.5 裁剪、摘要和排序

Context Builder 常见的三个动作是裁剪、摘要和排序。

裁剪决定哪些内容不进入上下文。它是最直接的 token 优化手段，但风险是把关键事实删掉。裁剪不能只按长度做，还要考虑任务目标、最近修改、文件类型、权限和历史重要性。

摘要把长内容压缩成短内容。摘要可以降低 token，但会引入信息损失。技术文档、代码 diff、工具输出和错误日志的摘要方式不应该相同。一个好的摘要应该保留可验证事实、关键标识符、失败原因和下一步所需约束，而不是只保留流畅自然语言。

排序决定模型先看到什么。LLM 对上下文位置敏感，重要内容如果被埋在中间或末尾，效果会下降。Context Builder 应该把任务目标、关键约束、最新状态和最相关证据放在模型最容易使用的位置。

7.6 Prompt Caching 与稳定前缀

很多 Agent 成本来自重复发送稳定内容：系统提示、工具定义、输出格式、项目级规则、术语约定。这些内容在多轮 Agent 循环中变化很少，适合成为稳定前缀。

Prompt Caching 的价值在于降低重复前缀的成本。它并不改变 Agent 的逻辑，但会改变系统设计：稳定内容应该尽量集中、顺序稳定、少做无意义改写；动态内容应该放在后面，并且尽量只包含本轮任务相关信息。

这和传统系统里的缓存友好设计类似。为了让缓存命中，系统需要稳定 key、稳定结构和可预测的变化范围。Agent Prompt 也一样。把每轮都变化的时间戳、随机描述和无关日志放进前缀，会降低缓存收益。

7.7 失败模式

Context Engineering 失败通常不是模型突然变差，而是数据加载策略出了问题。

第一，召回不足。关键文件、Memory 或工具结果没有进入上下文，模型只能猜。

第二，召回过度。太多无关内容进入上下文，模型被噪音干扰，成本上升。

第三，摘要失真。摘要丢掉约束、边界条件或错误细节，模型基于错误压缩结果继续推理。

第四，顺序错误。关键信息存在，但位置太差，模型没有有效使用。

第五，缓存失效。稳定前缀被频繁改写，导致 Prompt Caching 无法命中。

7.8 本章小结

本章把 Context Engineering 定位为 Query Optimization 问题。Agent 不应该追求无限上下文，而应该像数据库优化器一样选择、过滤、排序和压缩数据。

这个视角解释了为什么 AGENTS.md 重要。它不是普通说明文件，而是帮助 Agent 找到项目知识入口的索引结构。第 8 章会把 AGENTS.md 放到 Prompt Index 的框架下讨论。

第 8 章 AGENTS.md 与 Prompt Index

本章定位：解释为什么 AGENTS.md 更像项目级索引，而不是普通 README。

8.1 本章问题

Agent 进入一个项目时，面对的不是单个文件，而是整个工作目录：代码、文档、配置、测试、脚本、历史约定和隐含工作流。即使模型很强，如果每次都从零扫描整个项目，成本和延迟都会很高，结果也不稳定。

AGENTS.md 的价值就在这里。它不是给人看的普通说明文件，也不只是“给模型的提示”。从系统角度看，它更像项目级 Prompt Index：一个小入口，告诉 Agent 应该从哪里开始理解项目、哪些规则必须遵守、哪些文件是关键路径。

本章讨论 AGENTS.md 为什么像索引，以及一个好的 AGENTS.md 应该索引什么、不应该承载什么。

8.2 README、文档和 AGENTS.md 的区别

README 通常面向人类读者，介绍项目目标、安装方式和基本使用。详细文档面向维护者或用户，解释设计、API 和操作流程。AGENTS.md 面向 Agent，目标不是完整解释项目，而是帮助 Agent 快速定位相关知识。

这意味着 AGENTS.md 不应该重复所有文档。它应该像索引一样指出：构建命令在哪里，测试命令是什么，代码风格有什么硬约束，哪些目录属于核心模块，哪些文件不应该随便改，遇到某类任务应该先读哪里。

如果 README 是项目首页，AGENTS.md 更像查询入口和路由表。它不替代文档，而是帮助 Agent 决定要加载哪些文档。

8.3 AGENTS.md 作为 Prompt Index

索引的价值在于用小结构定位大内容。数据库索引不会保存整行所有信息，但它能帮助查询快速找到相关行。AGENTS.md 也类似：它不应该包含整个项目知识，但应该包含足够的路由信息，让 Agent 找到正确位置。

一个项目级 Prompt Index 至少应该回答几类问题：当前仓库的技术栈是什么？常用命令是什么？测试和格式化怎么跑？核心目录如何划分？任务相关文档在哪里？哪些约定比局部代码更重要？哪些操作有风险？

这些信息如果散落在多个文件里，Agent 每次都要重新探索。AGENTS.md 把这些入口集中起来，可以减少上下文加载成本，也能减少误读项目结构的概率。

8.4 好的 AGENTS.md 应该包含什么

内容类型	例子	作用
项目定位	这是 API 服务、前端应用、数据管	帮助 Agent 建立任务边界

	线还是文档项目	
关键目录	`src/`、`tests/`、`docs/`、`scripts/`	帮助 Agent 快速定位文件
常用命令	、格式化、建、生成 release	减少猜命令和跑错命令
代码/文档约定	命名、格式、术语、章节结构	保持修改风格一致
风险边界	不要改生成文件、不要重写某些章节、不要删除 release	避免破坏性操作
任务路由	修 API 先读哪里，写文档先读哪里	帮助 Agent 做上下文选择

这些内容应该简洁、稳定、可执行。AGENTS.md 不是长篇架构文档，而是高信号索引。

8.5 不应该放进 AGENTS.md 的东西

AGENTS.md 最大的风险是膨胀。如果它变成另一本手册，Agent 仍然需要读大量文本，索引价值就下降了。

不适合放进 AGENTS.md 的内容包括：完整业务背景、长篇教程、所有 API 细节、所有历史决策、可以从代码直接看出的重复信息、会频繁变化的临时任务列表。这些内容应该放在专门文档或 issue 里，再由 AGENTS.md 指向它们。

换句话说，AGENTS.md 应该保存“去哪找”和“必须遵守什么”，而不是保存“所有知识本身”。它的目标是提高 Context Builder 的命中率，而不是扩大 Prompt。

8.6 多层 AGENTS.md 与作用域

大型项目可能需要多个 AGENTS.md。根目录文件提供全局规则，子目录文件提供局部规则。比如后端、前端、移动端、文档、数据管线都可能不同命令和约定。

这和配置文件继承或路由表类似。全局规则适用于整个仓库，局部规则只在特定目录生效。Agent 在修改某个文件时，应该加载从根目录到目标目录路径上的相关 AGENTS.md，而不是只读一个全局文件。

作用域非常重要。如果所有局部规则都塞进根 AGENTS.md，文件会膨胀；如果只有局部规则没有全局规则，Agent 可能丢失项目级约束。多层索引能在两者之间取得平衡。

8.7 AGENTS.md 的失败模式

- 第一，过时。命令、目录或约定已经变了，但 AGENTS.md 没更新，Agent 会被错误索引误导。
- 第二，过长。AGENTS.md 变成百科全书，导致每次加载成本高，真正关键的规则被淹没。
- 第三，模糊。只写“保持代码质量”“注意测试”，没有具体命令和边界，Agent 无法执行。
- 第四，和源码冲突。AGENTS.md 说一种约定，代码里实际是另一种，Agent 不知道应该相信谁。
- 第五，没有任务路由。文件列了一堆规则，但没有告诉 Agent 遇到不同任务应该先读哪里。

8.8 本章小结

本章把 AGENTS.md 定位为 Prompt Index。它的作用不是替代 README 或完整文档，而是给 Agent 一个低成本、高信号的项目入口。

一个好的 AGENTS.md 应该帮助 Agent 做 Context Routing：从大量项目文件中找到本次任务需要加载的规则、文档和代码。第 9 章会继续讨论 Retrieval 和 Context Routing，说明检索不只是找相似文本，而是选择正确上下文路径。

第 9 章 RAG、Retrieval 与 Context Routing

本章定位：把 Retrieval 从“找相似文本”扩展为 Agent 的上下文路由问题。

9.1 本章问题

RAG 通常被解释为“先检索，再生成”。这个定义没有错，但对 Agent 系统来说太窄。真实 Agent 面对的不只是知识库文档，还包括项目文件、代码、Memory、工具结果、任务状态、issue、日志和外部系统。

因此，Agent 里的 Retrieval 不应该只理解成向量相似度搜索。它更像 Context Routing：根据任务类型、权限、状态和成本，决定应该从哪些信息源读取什么内容，并以什么形式放进模型上下文。

本章要回答的问题是：检索在 Agent 里到底承担什么职责？为什么“找相似文本”只是其中一部分？

9.2 RAG 是读取路径的一种

第 5 章已经说过，RAG 是从 Storage 到 Context 的一种读取路径。它适合处理大量非结构化文本，比如文档、知识库、手册、历史记录。检索系统找到相关片段，Context Builder 再把片段放进 Prompt。

但 Agent 的读取路径不止 RAG。查数据库、读文件、调用 API、读取 Memory、读取任务状态、查看工具日志，也都是读取路径。它们不一定使用向量检索，但都在回答同一个问题：本次模型调用需要哪些外部信息？

所以更准确的架构说法是：RAG 是 Context Routing 的一个实现手段，而不是整个上下文系统。

9.3 Context Routing 的输入

Context Routing 需要根据多个信号决定读取路径。

信号	例子	影响
任务类型	写代码、查事实、总结邮件、改文档、排查 bug	决定优先读代码、文档、Memory 还是工具结果
数据形态	结构化表、长文档、代码、日志、图片	决定使用 SQL、全文检索、向量检索还是文件读取
新鲜度	当前文件、历史 Memory、实时 API	决定是否相信缓存或必须重新读取
权限	用户授权、仓库权限、工具权限	决定哪些源可以访问
成本	token、延迟、API 调用成本	决定读取多少和是否压缩
	是否会影响生产系统、是否含隐私数据	决定是否需要人工确认或审计

这些信号说明，Retrieval 不是单纯的相似度排名。相似度只是一个分数，Context Routing 要做的是系统级选择。

9.4 向量检索的价值和边界

向量检索擅长从大量文本中找到语义相近内容。它适合查文档、找历史讨论、匹配相似问题、定位概念解释。对 Agent 来说，它可以显著降低“从零读项目”的成本。

但向量检索也有边界。第一，它可能召回看起来相关但实际上过期或错误的内容。第二，它不天然理解权限和任务状态。第三，它对结构化查询不一定比 SQL 更好。第四，它返回的是片段，不一定保留完整上下文和因果关系。

因此，向量检索应该和 metadata filter、时间戳、权限检查、文件路径、代码结构、任务状态结合使用。单纯“top-k 相似片段”很容易把错误上下文送进模型。

9.5 Hybrid Retrieval

生产 Agent 往往需要 Hybrid Retrieval。不同来源和检索方式互相补充：关键词检索适合精确标识符，向量检索适合语义相似，结构化查询适合状态和权限，文件路径适合项目结构，工具调用适合实时信息。

比如排查一个 bug，Agent 可能先用错误信息做关键词搜索，再用文件路径找到相关模块，再读取测试，再查最近修改，再从 Memory 中找项目约定。这里没有单一检索方法能解决全部问题。

Hybrid Retrieval 的关键是路由顺序。先读什么、后读什么、什么时候停止、什么时候升级到更贵的检索或模型调用，都会影响成本和质量。

9.6 Retrieval 结果不是最终 Context

检索返回的结果不能直接等同于最终 Context。检索结果还需要过滤、去重、排序、压缩和验证。

一个常见错误是把 top-k 片段原样塞进模型。这样可能包含重复内容、过期内容、冲突内容和无关内容。Context Builder 应该先判断哪些片段真正支持当前任务，再决定是否摘要、引用、保留原文或丢弃。

对于代码和工具输出尤其如此。代码片段需要保留函数名、文件路径和调用关系；日志需要保留时间、错误码和关键堆栈；工具输出需要保留可验证事实和副作用状态。不同内容需要不同压缩策略。

9.7 Context Routing 的失败模式

第一，路由错源。应该查数据库，却用了文档检索；应该读当前文件，却用了旧 Memory。

第二，召回偏差。检索结果语义相似，但和当前任务约束不匹配。

第三，权限越界。Agent 加载了当前用户不应该看到的信息。

第四，新鲜度错误。系统使用了缓存或旧索引，没有读取最新状态。

第五，停止条件不清。Agent 不断检索和追加上下文，成本上升但质量没有提高。

这些失败模式说明，Retrieval 需要和权限、状态、缓存、审计和 Planner 结合，而不是独立存在。

9.8 本章小结

本章把 RAG 和 Retrieval 放到 Context Routing 的框架下。RAG 是一种重要读取路径，但 Agent 的上下文系统还包括数据库、文件、Memory、工具结果、任务状态和日志。

真正有用的 Retrieval 不是只找相似文本，而是根据任务、权限、状态和成本选择正确的信息源，并把结果加工成模型真正需要的 Context。下一章会回到成本模型，继续讨论 Token Reduction、Distillation 和 Tiered Compute 的关系。

后续章节写作计划

第一部分已经完成前九章的基础架构视角。后续章节将沿着这条主线继续展开：Token Reduction 与 Distillation 的成本边界，Agent 作为生产系统的可靠性问题，多 Agent / 多租户调度，以及 Agent 安全和 Agent OS。

后续写作还需要补上三块更能体现工程差异化的内容。第一，Agent 安全不能只作为风险提示处理。OS 的安全模型会迁移到 Agent 系统里：Prompt Injection 更接近 exploit，工具调用需要权限边界，代码执行和外部系统访问需要 Sandbox。第二，并发调度是 OS 类比最直接的一部分，多 Agent、多租户、多任务队列和资源隔离，比单任务 Planner 更接近 Scheduler。第三，Agent 应该被当成生产系统来讨论：它需要 SLA、幂等、乐观锁、状态机、Retry、降级、升级、可观测、审计、回放和对账。这是本文区别于只讨论“能跑起来”的 OS-analogy 架构文章的重点。

1. 第 1-9 章已经建立从 LLM、Agent、Memory / Tool / Planner、Compute / Storage Separation、Stateless Agent 到 Context Engineering、Prompt Index 和 Context Routing 的基础架构主线。
2. 第 10 章将讨论 Token Reduction、Distillation、Small Model 和分层计算，重点拆开“减少 token 量”和“降低单 token 算力成本”这两条不同优化轴。
3. 第 11 章将把 Agent 当成生产系统来讨论，重点包括幂等、乐观锁、状态机、retry、降级、升级、可观测、审计、回放和对账。
4. 第 12 章将讨论 Multi-Agent、多租户和并发调度，把 Planner 与 Scheduler 的类比从单任务执行推进到资源竞争和任务编排。
5. 第 13 章将补上 Agent 安全模型，重点包括 Prompt Injection 作为 exploit、工具权限边界、Sandbox、最小权限和审计。
6. 第 14 章将回到 Agent Operating System，把 Memory、Tool、Planner、安全、调度和生产可靠性放回统一系统视角。